

Exercise 1: Setting Up the Project and Your First Signals

Context: Liya is a student at CoTBE. She opens the TMS in her browser expecting to see her dashboard: her name, how many credits she has earned, and whether she is eligible for graduation. Right now, there is nothing. You are going to build that dashboard using Angular's reactive **Signals** architecture.

[!NOTE]

Step 1: Create the Angular Project

Open a terminal, navigate to your workspace directory, and initialize a modern standalone Angular application:

```
ng new tms-client --style=scss --ssr=false --standalone --routing
```

Angular CLI Architecture Configuration Flags Explained:

- **tms-client**: The name of the project folder containing our Training Management System frontend.
- **--style=scss**: Configures the preprocessor format to SCSS so you can leverage nested rules and cleaner design variables.
- **--ssr=false**: Disables Server-Side Rendering. TMS is a login-protected dashboard that does not require public SEO indexing, reducing operational complexity.
- **--standalone**: Eliminates the legacy **NgModule** system (**app.module.ts**) in favor of tree-shakable standalone components.
- **--routing**: Scaffold an explicit routing configuration file to manage client-side state transfers between workspace screens.

Once compilation completes, enter the workspace and launch your editor:

```
cd tms-client  
code .
```

[!NOTE]

Step 2: Configure the Application Shell

Open **src/app/app.config.ts**. Replace the boilerplate text with this unified bootstrapper to initialize zoneless change detection and global injection providers:

```
import { ApplicationConfig, provideZonelessChangeDetection } from  
"@angular/core";  
import { provideRouter, withComponentInputBinding } from  
"@angular/router";  
import { provideHttpClient } from "@angular/common/http";  
import { routes } from "./app.routes";
```

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideZonelessChangeDetection(),
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(),
  ],
};
```

Application Core Provider Subsystems:

- `provideZonelessChangeDetection()`: Drives high-performance screen updates natively using Signals, completely removing the heavy Zone.js macro-task evaluation overhead.
- `provideRouter(..., withComponentInputBinding())`: Resolves system URL routes and maps parameters directly into component inputs automatically.
- `provideHttpClient()`: Plugs in the low-level asynchronous networking client to enable backend communications with your .NET Core Web API.

[!NOTE]

Step 3: Generate the Dashboard Component

Execute the schematic generator ensuring explicit file-suffix preservation for proper feature colocation:

```
ng generate component features/student-dashboard --type=component
```

Generated Workspace Deliverables:

- `student-dashboard.component.ts`: The TypeScript logic container defining your state engines and computations.
- `student-dashboard.component.html`: The structural markup blueprint declaring what your student sees.
- `student-dashboard.component.scss`: The local stylesheet isolated explicitly to this feature's layout boundaries.

[!NOTE]

Step 4: Build the Signal State

Open `src/app/features/student-dashboard/student-dashboard.component.ts`. Replace its contents with this reactive script tracking credit status mutations:

```
import { Component, signal, computed } from "@angular/core";

@Component({
  selector: "app-student-dashboard",
  standalone: true,
```

```

imports: [],
templateUrl: "./student-dashboard.component.html",
styleUrl: "./student-dashboard.component.scss",
})
export class StudentDashboardComponent {
  // signal() wraps primitive variables into reactive data cells
  // observed by Angular
  studentName = signal("Liya Kebede");
  earnedCredits = signal(45);

  // computed() maps a read-only dependency link that recalculates
  // only when inner signals emit mutations
  graduationStatus = computed(() =>
    this.earnedCredits() >= 120 ? "Eligible for Graduation" : "In
    Progress",
  );

  // Triggering .update() captures current states and computes
  // structural modifications smoothly
  registerForClass() {
    this.earnedCredits.update((c) => c + 3);
  }
}

```

[!NOTE]

Step 5: Build the Template Canvas

Open `src/app/features/student-dashboard/student-dashboard.component.html`.
Strip the boilerplate file text and insert your data bindings:

```

<div class="dashboard">
  <h1>Welcome, {{ studentName() }}</h1>
  <p>Credits Earned: {{ earnedCredits() }}</p>
  <p>Graduation Status: {{ graduationStatus() }}</p>
  <button (click)="registerForClass()">
    Register for a Class (+3 credits)
  </button>
</div>

```

Template Integration Mechanisms:

- `{{ earnedCredits() }}`: Double curly braces read values out into your layout. Parentheses are required to evaluate and extract the value from the Signal container.
- `(click)="registerForClass()"`: Parentheses denote an outward event binding. When clicked, it hits the backing controller method without raw DOM selectors.

[!NOTE]

Step 6: Configure Client-Side Routing

Open `src/app/app.routes.ts` and set up your routing tables using asynchronous code-splitting lazy loading flags:

```
import { Routes } from "@angular/router";

export const routes: Routes = [
  {
    path: "dashboard",
    loadChildren: () =>
      import("../features/student-dashboard/student-
      dashboard.component").then(
        (m) => m.StudentDashboardComponent,
      ),
  },
  { path: "", redirectTo: "dashboard", pathMatch: "full" },
];
```

Finally, open `src/app/app.component.html`, clear the default template landing layout, and insert the framework anchor tag:

```
<router-outlet />
```

Note: `<router-outlet>` acts as a placeholder telling the router where to insert the component matching the current browser URL path.

[!NOTE]

Step 7: Launch the Application

Start the local development server by executing this command in your terminal:

```
ng serve
```

Open your browser and navigate to `http://localhost:4200/dashboard`.

Expected Verification Behavior: The dashboard loads successfully displaying Liya's name, her initial credits (45), and her graduation status (In Progress). Each click on the registration button increments the credit score by 3. The graduation status remains In Progress until the credits counter reaches 120, at which point it automatically flips to Eligible for Graduation.

Troubleshooting & Common Edge Cases

Problem	Likely Cause	Fix
---------	--------------	-----

Problem	Likely Cause	Fix
Blank page at /dashboard	Missing <code><router-outlet /></code> tag placeholder inside <code>app.component.html</code> .	Open your <code>app.component.html</code> file and ensure it contains exactly <code><router-outlet /></code> .
{{ earnedCredits }} shows [object Object]	Omitted the evaluation parentheses from the signal variable call inside your HTML markup template.	Update the string interpolation expression syntax to call the function: <code>{{ earnedCredits() }}</code> .
"Cannot find module" compiler error	Typo or incorrect relative path declaration inside the route definition layout block.	Check your relative file import paths inside <code>app.routes.ts</code> to ensure they match your folder tree.
Port 4200 is already in use	A dangling node process or another local application instance is currently binding to that default port.	Launch the application on an alternative port using: <code>ng serve --port 4300</code> , or terminate the blocking process.

Checkpoint 1 Achieved